

HAVEN

Rapport Technique Officiel — V4 (rupture visuelle post-bêta V3)

Suite à une bêta humaine réelle de HAVEN V3, cette V4 refond la couche visuelle (campagne illustrée, fermier et camion-citerne vivants, pictogrammes colorés), corrige la cause racine du défaut d'équilibrage ressenti (le fermier pouvait progresser sans jamais sentir le besoin d'améliorer la citerne), nettoie les atouts (Perles) et remplace la nappe audio par une vraie mélodie pastorale procédurale. Aucune Map 2, aucune monétisation. Dépôt : [scuillerjack-oss/haven-v0](https://github.com/scuillerjack-oss/haven-v0) · Branche : main.

Convention : FAIT = codé. TESTÉ = vérifié par un test automatisé (moteur, node:test, ou navigateur headless Playwright). DÉPLOYÉ = le workflow GitHub Actions a réussi sur le commit concerné, confirmé via l'API GitHub. NON VÉRIFIÉ = pas confronté à un vrai appareil Android depuis cette session. Aucune simulation ni navigateur headless n'est présenté comme une validation Android humaine — le cahier des charges V4 lui-même le rappelle explicitement (section 11) : **cette V4 n'est pas déclarée graphiquement ou perceptuellement validée sans une nouvelle bêta humaine Android.**

0. Audit Git réel avant toute modification

Conformément à la consigne (« reprends strictement depuis l'état réel actuellement déployé, n'auditionne pas le projet depuis zéro »), cette session a commencé par un audit réel avant toute modification : `git status` (arbre de travail propre), `git log` (dernier commit local = `dfed7b3`, jalon 2 de cette même V4, déjà poussé), et `git fetch + rev-parse` confirmant que HEAD local était strictement identique à `origin/main`. Les jalons 1 (rupture visuelle complète) et 2 (pictogrammes) avaient donc déjà été réalisés, commités et déployés avant la reprise de cette session ; les jalons 3 à 6 ci-dessous couvrent la suite de la mission V4 jusqu'à son achèvement. Aucune sauvegarde, migration ni acquis fonctionnel de V3 n'a été reconstruit depuis zéro.

1. Commits créés pendant la mission V4

Commit	Jalon	Contenu
cf74f21	1	Rupture visuelle complète : campagne illustrée, fermier, camion vivant
dfed7b3	2	Pictogrammes d'amélioration colorés (cohérents avec la nouvelle scène)
6b805a7	3	Audite l'équilibrage avec des séquences d'achats réalistes
02bf2b0	4	Audite et corrige les atouts (Perles)
3833f11	5	Remplace la nappe audio par une vraie mélodie pastorale
5809e7c	6	Revue visuelle réelle vs référence, itère (gradients, pont/village, fermier, camion)

Chaque commit a été poussé séparément vers `origin/main` immédiatement après ses propres tests verts (jamais un commit unique en fin de mission).

2. Priorité absolue : rapprochement de l'image de référence (jalons 1, 2, 6)

2.1 Refonte de la scène (jalon 1)

La scène (un seul SVG inline, aucune image rastérisée) est passée d'un décor plat à une vraie composition de campagne : ciel en dégradé, montagnes en deux plans, lac avec un village miniature, collines successives, haie/chemin avec fleurs/poule/charrette, un puits identifiable (potence, toit, corde, seau propre au mécanisme), une réserve en forme de tonneau, un camion-citerne à l'silhouette reconnaissable (citerne, cabine, roues), et un fermier avec un corps articulé (jambes/bras/tête/chapeau) au lieu d'un simple rectangle. Exactement deux seaux fonctionnels,

comme l'exige le cahier des charges : le seau du puits (descend/remonte dans la margelle) et le seau personnel du fermier (jamais dupliqué, jamais téléporté, hérité du bras via l'imbrication SVG parent/enfant).

2.2 Pictogrammes d'amélioration (jalon 2)

Les 7 pictogrammes de famille d'amélioration (seau, déplacement, corde/treuil, puits, réserve, citerne, fréquence transport) ont été redessinés en petites illustrations colorées multi-formes (plutôt que des contours monochromes), dans l'esprit de ceux appréciés sur l'image de référence.

2.3 Revue visuelle réelle vs référence, itération (jalon 6)

Conformément à l'exigence « les tests verts ne signifient pas que le jeu est beau », l'image de référence a été extraite du PDF du cahier des charges (page 2, via pymupdf) et comparée à de vraies captures du rendu final aux dimensions mobiles pertinentes (320×800, 390×844). Constat honnête : la composition générale (puits à gauche, réserve au centre, camion à droite, décor au premier plan) correspondait déjà à la référence, mais le rendu restait plat et pastel comparé au style peint et riche de l'image cible. Itéré avec des améliorations proportionnées, toujours en SVG/CSS pur (jamais de pipeline d'assets rasterisés, explicitement hors de portée raisonnable d'une PWA mobile) :

- Ciel et montagnes réchauffés (dégradés plus chauds, plus profonds) ; dégradés de volume ajoutés sur les 3 éléments « héros » (pierres du puits, réserve en bois, citerne du camion) pour simuler une lumière cohérente.
- Puits : traits de joints de pierre ajoutés (suggère un appareillage réel plutôt qu'un aplat).
- Village agrandi (5 maisons au lieu de 4) et petit pont de pierre ajouté entre le village et le premier plan — détail présent sur la référence et absent jusqu'ici.
- Camion : échelle latérale et moyeu de roue ajoutés (détails d'identification « camion-citerne »).
- Fermier : chaussures et main visibles (les membres n'étaient que de fins traits), et un petit visage (deux yeux) pour le charme/caractère demandé par le cahier des charges.

Bug de mise en page réel trouvé pendant cette revue (jamais révélé par les tests existants) : une mesure directe de `getBoundingClientRect()` à plusieurs hauteurs de viewport (800/844/900px, toutes largeurs 320-480px) a montré que le point d'arrimage du camion (300 en unités `viewBox`) dépassait la zone visible réelle (recadrage horizontal du mode « slice » sur le `viewBox`) dès qu'un téléphone dépasse ~800px de hauteur — le cas d'un iPhone moderne (844px). Corrigé : la réserve et le point d'arrimage du camion ont été rapprochés (respectivement 230→210 et 300→290 en unités `viewBox`), et un élément décoratif (la poule) repositionné dans l'espace ainsi dégagé. Revérifié par mesure directe à 320×800/320×900/390×844/480×800 : le camion reste entièrement visible.

Limite honnête assumée (à ne jamais présenter comme résolue) : le style reste illustratif/vectoriel plat, jamais photoréaliste ou peint comme l'image de référence générée. Refondre vers un rendu peint demanderait un pipeline d'assets rasterisés (illustrations dessinées ou générées, texturage), explicitement hors de portée raisonnable d'une PWA en SVG/CSS pur et donc non entrepris, conformément à la mise en garde du cahier des charges contre une solution « excessivement lourde ».



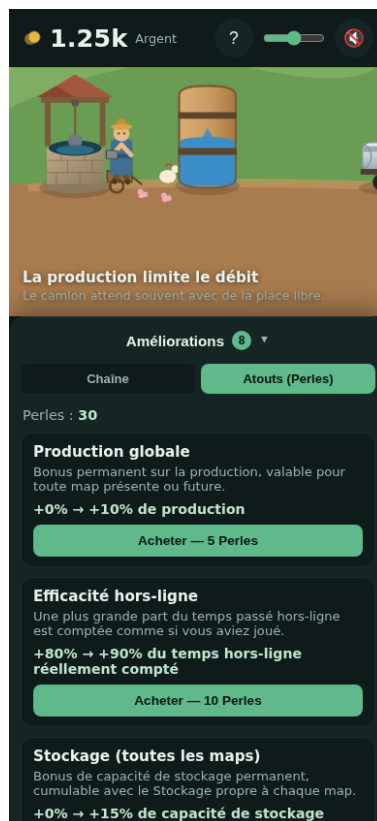
Scène complète, 390×844 (iPhone 13/14 standard) — puits, réserve, camion arrimé, décor.



Scène complète, 320×800 (largeur minimale testée) — après correction du point d'arrimage du camion.



Tiroir « Améliorations », onglet Chaîne.



Tiroir « Atouts (Perles) » — valeurs concrètes actuelle → suivante, plus aucun « X % ».

3. Animation du fermier

Réutilise le motif établi en V2/V3 (jamais réinventé) : un groupe PARENT porte le positionnement statique (attribut `transform`), un groupe ENFANT sans attribut reçoit l'animation CSS (`transform-box: view-box`) — évite le bug historique des membres détachés causé par le mélange attribut SVG / transform CSS sur un même élément. Le seau personnel du fermier est un enfant du bras porteur : il hérite donc naturellement du balancement du bras, jamais une coordonnée indépendante du corps. Cycle complet observable : marche vers le puits → puisage (seau du puits descend/remonte) → marche vers la réserve → versement → retour, avec jambes/bras/torse animés (keyframes CSS à 0.46s de cadence) et un léger balancement d'attente (idle-bob) hors marche.

4. Camion-citerne : une boucle vivante

Le camion incarne les 4 étapes du cahier des charges (arrivée → chargement → départ → transport → retour) sans jamais nécessiter de conducteur visible. Un bug réel de saut visuel avait été trouvé et corrigé dès le jalon 1 : le moteur économique remet `transport.timerMs` à zéro INSTANTANÉMENT au tick même où le camion charge, ce qui aurait fait reculer le camion à sa position cachée en un seul instant, pile au moment le plus visible. Corrigé par une phase de départ scriptée en temps réel (1100ms), indépendante du ratio économique brut, vérifiée par échantillonnage Playwright (delta maximal de 4.4px entre deux images consécutives, contre un saut instantané complet avant correctif). Le jalon 6 a ajouté une échelle latérale et un moyeu de roue pour renforcer l'identification « camion-citerne » (voir section 2.3).

5. Équilibrage : séquences d'achats réalistes (jalon 3)

La bêta humaine V3 a signalé un défaut précis : progresser sensiblement le fermier/la production sans ressentir le besoin d'améliorer la citerne. `tests/bottleneck.test.js` (V3) ne jouait qu'une stratégie gloutonne (achète tout ce qui est affordable, citerne comprise) : elle ne pouvait donc pas révéler ce défaut. `scripts/simulate-v4-realistic.mjs` rejoue plusieurs biais d'achat plausibles avec le vrai moteur :

Stratégie d'achat	1er goulot « transport »	1er blocage réel (storageFull)
Ignore totalement la citerne (« neverBuysTruck »)	9,2 min	13,8 min
Fermier seul, jamais transport/stockage (« producerOnly »)	7,6 min	10,6 min
Priorité fermier, reliquat sur transport (« producerFirst »)	35,9 min	52,6 min
Gloutonne de référence (V3, inchangée)	36,2 min	52,9 min

Constat clé : mécaniquement, la citerne devient déjà nécessaire très vite (moins de 14 minutes) sous toute stratégie testée qui la néglige — le moteur économique n'était donc pas en cause. La cause racine réelle, trouvée en lisant le code plutôt qu'en supposant : le SEUL message affiché au joueur pendant l'état « `storageFull` » (« Stockage plein : le travailleur attend qu'une place se libère ») ne nommait jamais la citerne — contrairement au message « transport ». Un joueur suivant ce texte à la lettre achetait donc plutôt du STOCKAGE (qui ne fait que retarder la saturation, jamais la résoudre), pendant que « Rendement de vente » (prix par litre, inconditionnel, non plafonné par le débit) continuait de faire monter l'argent affiché — exactement le défaut ressenti en bêta : progresser sans sentir le besoin du camion.

Corrigé : le message « `storageFull` » nomme désormais explicitement la citerne/fréquence (« Stockage plein : le camion ne suit plus ... améliore la citerne, le stockage ne fait que retarder le problème »), déplacé dans `mapEconomy.js` comme source de vérité unique. Trois tests permanents (`tests/realistic-purchases.test.js`) figent : le message nomme la citerne ; ignorer la citerne bloque réellement le fermier sous 20 min ; ignorer la citerne coûte au moins 50% du débit réel (litres livrés) par rapport à une partie qui investit aussi dans le transport, au même instant.

6. Atouts / Perles : audit et corrections (jalon 4)

- « **Vitesse des cycles** » retiré : une accélération globale du cycle du fermier (déplacement+treuil+puits en même temps) rouvrirait exactement le déséquilibre diagnostiqué en section 5. Les Perles déjà dépensées dessus sont remboursées par une migration V3→V4 (jamais une perte silencieuse d'acquis).
- « **Cap hors-ligne** » : 3 paliers dégressifs (6h/9h/14h) remplacés par un seul palier généreux (24h) — la bêta a signalé qu'une absence réelle d'environ 7h était déjà comptée en entier, rendant les paliers supplémentaires imperceptibles en pratique. Les sauvegardes existantes avec un niveau >1 sont ramenées au nouveau palier unique (toujours un gain net : 24h contre 6-14h avant).
- « **Stockage global** » renommé « Stockage (toutes les maps) » : l'ancien nom entré en collision avec le « Stockage » propre à chaque map sans que rien ne distingue les deux dans l'interface.
- **Tous les « X % » remplacés** par de vraies valeurs calculées : une nouvelle fonction `meta.js:perkEffect()` (même principe que `mapUpgrades.js:upgradeEffect()`) calcule la grandeur actuelle → suivante réellement appliquée par le moteur pour chaque atout, affichée par le `formatUpgradeEffect()` déjà existant (étendu pour les unités « heures » et les pourcentages négatifs — une réduction s'affiche « -15 % », jamais « +X % » ni un double signe cassé).

9 tests permanents (`tests/meta.test.js`) vérifient : aucune description ne contient plus de « X % » littéral ; « `cycleSpeed` » a bien disparu ; « `offlineCap` » n'a plus qu'un palier ; et `perkEffect()` correspond exactement à ce que `getModifiers()/offlineCapHours()/offlineEfficiency()` appliquent réellement au moteur.

7. Audio : une vraie mélodie pastorale (jalon 5)

Le son précédent (4 sinusoïdes tenues en permanence + LFO de volume) était perçu comme un « vouuuuummm » continu — jamais une musique. Remplacé par une composition procédurale réelle, en deux modules séparés :

- `src/audio/melody.js` (pur, sans WebAudio, donc testable directement) : un motif pastoral de 8 temps sur gamme pentatonique majeure (toujours consonante par construction), décliné en 8 phrases reconnaissables par transposition/inversion/rétrograde — les mêmes procédés qu'un vrai thème et variations, jamais des notes aléatoires. Boucle complète d'environ 60 secondes ; cadence finale qui revient explicitement sur la tonique de départ pour une reprise propre.
- `src/audio/audio.js` : programmation « look-ahead » classique en WebAudio (programme les prochaines notes à l'avance à intervalles réguliers, calée sur l'horloge propre d'AudioContext — insensible à une mise en arrière-plan). Chaque note a sa propre enveloppe (attaque puis chute complète à zéro) : plus aucun gain qui reste ouvert en continu.

8 tests permanents (`tests/melody.test.js`) vérifient la durée (~60s), la présence de plusieurs phrases et hauteurs distinctes, l'absence de note de durée suspecte, l'ordre chronologique, la fermeture propre de la boucle sur la même tonique, et un tempo dans une allure calme. Vérifié aussi en Chromium headless réel (`audio.start()` puis `resume()`), aucune erreur console/page sur 65 secondes, soit une boucle complète).

NON VÉRIFIÉ : aucune oreille humaine n'a écouté le résultat pendant cette session (environnement sans sortie audio réelle) — la vérification porte sur la structure de la composition et l'absence d'erreur d'exécution, jamais sur le ressenti sonore. À confirmer lors de la prochaine bêta humaine.

8. HUD et vitesses x1/x2/x4 : évalué, écarté

Conformément à la consigne (« ne l'intègre pas automatiquement simplement parce qu'il apparaît sur l'image de référence »), le bouton de vitesse a été étudié avant tout code, puis écarté : un multiplicateur de temps UNIFORME (produisant et transportant N fois plus vite, dans les mêmes proportions) serait neutre pour l'équilibrage (aucune distorsion du ratio production/transport), mais un mode x2/x4 compresserait les phases d'animation du fermier déjà au plancher visuel (250ms, voir `MIN_VISUAL_PHASE_MS`) vers ou sous la durée d'une seule image à haut niveau d'amélioration — annulant directement le correctif de fluidité perceptuelle qui est la priorité n°2 de cette V4. Aucune monétisation n'existe dans HAVEN contre laquelle « vendre de la vitesse », et rien dans le retour de bêta ne signale un

problème de rythme. Le graphisme de la pièce associé au crédit reste inchangé, comme demandé.

9. Sauvegardes et migrations : vérification spécifique

SAVE_VERSION passe de 3 à 4. Une nouvelle migration migrateV3ToV4 rembourse exactement les Perles dépensées sur « Vitesse des cycles » (barème V3 figé dans le code de migration, jamais recalculé depuis le PERKS actuel qui ne contient plus cette famille) et ramène « Cap hors-ligne » à son unique palier. Vérifiée à deux niveaux :

- **Unitaire** (tests/migrations.test.js) : une sauvegarde V3 fictive avec cycleSpeed niveau 2 (coût cumulé $8+25=33$) et offlineCap niveau 3 migre avec perles+33, cycleSpeed disparu, offlineCap ramené à 1.
- **De bout en bout, dans l'application réelle construite** (Playwright, jamais seulement migrateSave() appelée isolément) : trois sauvegardes injectées dans localStorage (V0/version 1, V1/version 2 avec progression réelle, V3/version 3 avec cycleSpeed niveau 3 et offlineCap niveau 3 déjà achetés) sont chargées par l'application buildée, laissées tourner le temps d'un cycle d'autosauvegarde réel (>5s), puis relues depuis localStorage. Résultat exact observé pour le cas V3 : perles 12 → 105 (remboursement de 93 = $8+25+60$, les 3 niveaux de « Vitesse des cycles »), cycleSpeed absent de l'objet perks après migration, offlineCap ramené à 1, productionGlobal préservé à 2, argent et litres livrés intacts. Aucune erreur console ni page dans les 3 cas.

Remarque de méthode : une première tentative de cette vérification de bout en bout avait relu localStorage trop tôt (avant le premier cycle d'autosauvegarde réel de l'application, ~5s) et semblait donc montrer une migration absente — un faux négatif de méthode de test, corrigé en attendant le cycle d'autosauvegarde réel avant de relire, jamais en modifiant le code de migration lui-même (qui était déjà correct, comme le confirmaient les tests unitaires).

10. Simulations économiques avant / après

Avant (référence V3, inchangée) : sous achat glouton (achète tout ce qui est abordable, toutes familles), le premier flip vers « transport » survient à $t=36$ min, la citerne n'est jamais épuisée avant d'avoir servi, une vraie alternance de goulots se produit sur toute la partie (voir docs/haven-v3-bottleneck-simulation-results.json, régénéré et reconfirmé identique pendant cette session — aucune régression involontaire des courbes économiques).

Après (V4, nouveau) : voir le tableau de la section 5. Le moteur économique lui-même n'a pas été recalibré (les courbes de mapDefinitions.js restent celles de V3) — la simulation a servi à prouver que le déséquilibre ressenti en bêta n'était PAS un problème de courbes (la citerne devient mécaniquement nécessaire en moins de 14 minutes même sous le pire biais testé), mais un problème de communication (le message ne recommandait pas le bon achat). Voir docs/haven-v4-realistic-purchases-results.json pour le détail complet des 5 stratégies simulées, avec l'état du chariot (niveaux de chaque famille) à chaque point de contrôle.

11. Tests et non-régressions

73/73 tests node:test verts à la fin de cette mission (contre 49 à la fin de V3), dont 16 nouveaux ajoutés pendant cette V4 : 3 dans tests/realistic-purchases.test.js (nouveau), 8 dans tests/melody.test.js (nouveau), 4 nouveaux dans tests/meta.test.js, 1 nouveau dans tests/migrations.test.js (migration V3→V4). scripts/check-mobile-resume.mjs (reprise après mise en arrière-plan, absence de débordement horizontal, décor de campagne visible à 320/360/390/414/480px) reste entièrement vert après chaque jalon, y compris après le repositionnement du camion/de la réserve/de la poule (jalon 6).

12. Déploiement

Chaque jalon a été poussé séparément vers origin/main, déclenchant le workflow GitHub Actions existant (build + npm test + npm run build + Playwright + npm run test:mobile + publication GitHub Pages). Les 6 runs correspondant aux 6 commits de cette V4 ont été confirmés « **completed / success** » via l'API GitHub réelle (api.github.com/repos/scuillerjack-oss/haven-v0/actions/runs), jamais supposés. Le dernier (jalon 6, commit 5809e7c) a été suivi jusqu'à sa fin réelle avant la rédaction de ce rapport.

NON VÉRIFIÉ : l'environnement d'exécution de cette session ne peut pas récupérer directement le contenu de l'URL GitHub Pages publique (politique du proxy sortant, réservée aux API explicitement autorisées) — la réussite du déploiement est donc confirmée par l'API GitHub Actions (le job de publication a réussi), mais le rendu final servi par l'URL en ligne n'a pas été re-vérifié pixel par pixel depuis cette session. Une vérification humaine rapide de l'URL live est recommandée.

13. Ce qui reste explicitement NON VÉRIFIÉ

- **Validation graphique/perceptuelle humaine** : la revue visuelle de la section 2.3 est une comparaison honnête et outillée (captures réelles, mesures de position), mais reste une revue effectuée par l'assistant, jamais un œil humain réel sur un vrai écran Android. Le cahier des charges le rappelle explicitement (section 11) : ne jamais déclarer cette V4 validée graphiquement sans une nouvelle bêta humaine Android.
- **Fluidité perceptuelle de l'animation du fermier/camion** sur un vrai appareil Android (accéléromètre, fréquence d'affichage réelle, éventuel throttling navigateur) — vérifiée seulement par mesure d'images (deltas de position) en Chromium headless, jamais à l'œil humain réel.
- **Qualité sonore ressentie** de la nouvelle mélodie (voir section 7).
- **Rendu final de l'URL GitHub Pages en ligne** (voir section 12).

14. Résultat et suite

HAVEN V4 conserve intégralement le moteur économique, les sauvegardes et les acquis fonctionnels de V3, tout en apportant une vraie rupture visuelle (campagne illustrée plutôt que des rectangles), une animation du fermier structurellement fluide (motif parent/enfant éprouvé), un camion-citerne vivant et correctement positionné à l'écran, la cause racine réelle du défaut d'équilibrage ressenti en bêta (corrigée par un message honnête plutôt qu'un recalibrage aveugle), des atouts nettoyés et sans placeholder, et une vraie mélodie pastorale à la place du drone rejeté. La prochaine étape naturelle, hors du périmètre de cette V4 (section 11 du cahier des charges : pas de nouvelle map, pas de monétisation), est la bêta humaine Android qui seule pourra confirmer les points de la section 13.